

Белорусский государственный технологический университет
Факультет информационных технологий
Кафедра информационных систем и технологий

Лабораторная работа 1

По дисциплине «Проектирование и разработка баз данных
интернет-приложений»

На тему «Проектирование базы данных»

Выполнил:

Студент 3 курса 6 группы

Павлович Ян Андреевич

Преподаватель: Нистюк О.А.

Минск 2026

Оглавление

Введение.....	3
1 Описание интернет-приложения.....	4
2 Целевая аудитория и стратегии использования.....	4
3 Роли для базы данных.....	4
4 UML.....	5
5 Сущности и их атрибуты.....	6
6 Преобразование сущностей в таблицы.....	8
7 Логическая схема базы данных.....	8
8 Проверка нормальных форм.....	9
9 Денормализация.....	10
10 Оценка сроков проекта.....	10
11 Требуемые ресурсы.....	11
Заключение.....	12

Введение

В условиях цифровизации управления и роста объема информационных потоков в организациях особую значимость приобретает автоматизация процессов постановки и контроля исполнения задач. Традиционные методы управления поручениями, основанные на устной коммуникации или разрозненных инструментах учета, не обеспечивают необходимого уровня прозрачности, оперативности и контроля. Это приводит к нарушению сроков исполнения, снижению ответственности сотрудников и усложнению анализа эффективности деятельности подразделений.

Одним из ключевых инструментов повышения управляемости является внедрение специализированных интернет-приложений, обеспечивающих централизованный учет задач, распределение ролей и контроль выполнения поручений. Основой любой подобной системы выступает корректно спроектированная база данных, определяющая структуру хранения информации, механизмы взаимосвязи объектов и принципы обеспечения целостности данных. От качества проектирования базы данных напрямую зависят надежность системы, ее масштабируемость и возможность дальнейшего функционального расширения.

Проектирование базы данных включает анализ предметной области, выявление сущностей и их атрибутов, определение логических связей, формирование концептуальной и логической моделей, а также проверку структуры на соответствие нормальным формам. Грамотная нормализация позволяет минимизировать избыточность, устранить аномалии обновления и обеспечить согласованность данных при многопользовательской работе.

Целью данной лабораторной работы является разработка логической модели базы данных интернет-приложения «Контроль исполнения», предназначенного для автоматизации процессов постановки, мониторинга и анализа выполнения задач в организации. В рамках работы осуществляется формализация требований, построение UML-диаграммы, выделение сущностей предметной области, определение их атрибутов и связей, преобразование модели в реляционную структуру таблиц, а также проверка соответствия разработанной схемы нормальным формам вплоть до четвертой нормальной формы.

Результатом работы должна стать структурированная и логически непротиворечивая модель базы данных, пригодная для последующей реализации в современных системах управления базами данных и интеграции в архитектуру web-приложения.

1 Описание интернет-приложения

Разрабатываемое интернет-приложение ориентировано на централизованный учет и контроль выполнения задач внутри организации. Система поддерживает полный жизненный цикл поручения: от постановки задачи и назначения исполнителя до фиксации результатов и анализа исполнения.

Интернет-приложение «Контроль исполнения» предназначено для:

1. постановки поручений;
2. назначения ответственных исполнителей;
3. отслеживания сроков выполнения;
4. контроля статусов задач;
5. формирования отчетов;
6. хранения истории изменений.

Функциональность приложения охватывает создание задач, управление сроками, отслеживание статусов, хранение истории изменений и формирование отчетности. Архитектурно система предполагает использование web-интерфейса и реляционной базы данных, обеспечивающей централизованное хранение информации и многопользовательский доступ.

2 Целевая аудитория и стратегии использования

Система ориентирована на использование в организациях с иерархической структурой управления. Разграничение ролей пользователей позволяет обеспечить безопасность данных и корректное распределение полномочий.

Основные категории пользователей:

1. руководители подразделений;
2. исполнители;
3. администраторы системы.

Стратегии использования:

1. руководитель создает задачу и назначает исполнителя;
2. исполнитель получает уведомление и обновляет статус;
3. руководитель контролирует сроки и просматривает отчеты;
4. администратор управляет пользователями и структурой.

Таким образом, база данных должна обеспечивать поддержку различных пользовательских сценариев без нарушения целостности информации.

3 Роли для базы данных

Разделение ролей является важным элементом проектирования системы, поскольку напрямую связано с механизмами разграничения доступа. Каждая роль определяет набор допустимых операций и уровень

взаимодействия с данными. Распределение ролей и полномочий представлены в таблице 3.1.

Таблица 3.1 – Роли и полномочия

Роли	Полномочия
Администратор	Управление пользователями, ролями, структурой
Исполнитель	Создание задач, назначение исполнителей, контроль
Руководитель	Просмотр задач, изменения статуса, добавления комментариев

Реализация ролевой модели в базе данных позволяет централизованно управлять правами доступа и обеспечивает соответствие принципу минимально необходимого уровня привилегий. Это повышает безопасность системы и упрощает администрирование.

Дополнительно ролевой подход способствует формализации бизнес-процессов и обеспечивает четкое разграничение ответственности между участниками системы. Закрепление полномочий на уровне базы данных позволяет предотвратить несанкционированные операции, ограничить доступ к критически важной информации и обеспечить контроль действий пользователей. Таким образом, ролевой механизм выступает не только инструментом безопасности, но и средством структурирования организационной логики приложения.

4 UML

UML-диаграмма отражает структурную модель предметной области и визуализирует взаимосвязи между основными сущностями системы. Использование унифицированного языка моделирования (Unified Modeling Language) позволяет формализовать требования к системе и обеспечить однозначность интерпретации структуры данных всеми участниками проекта. Применение UML снижает риск неоднозначного понимания логики предметной области и упрощает переход от аналитического этапа к проектированию базы данных. Рисунок UML-диаграммы представлен на рисунке 4.1.

UML является стандартизированным средством объектно-ориентированного моделирования и широко применяется при проектировании информационных систем различного уровня сложности. В контексте разработки базы данных диаграммы позволяют структурировать объекты предметной области, определить их атрибутивный состав и установить типы связей между ними. Это особенно важно при разработке систем с несколькими взаимосвязанными сущностями, где необходимо учитывать иерархические и ассоциативные зависимости.

На этапе моделирования UML-диаграмма выполняет функцию формализованного описания логики функционирования системы. Она позволяет выявить потенциальные противоречия в структуре данных ещё до

этапа физической реализации в СУБД. Кроме того, визуальное представление облегчает анализ корректности распределения атрибутов по сущностям и помогает определить необходимость введения дополнительных таблиц и связей. UML диаграмма представлена на рисунке 4.1.

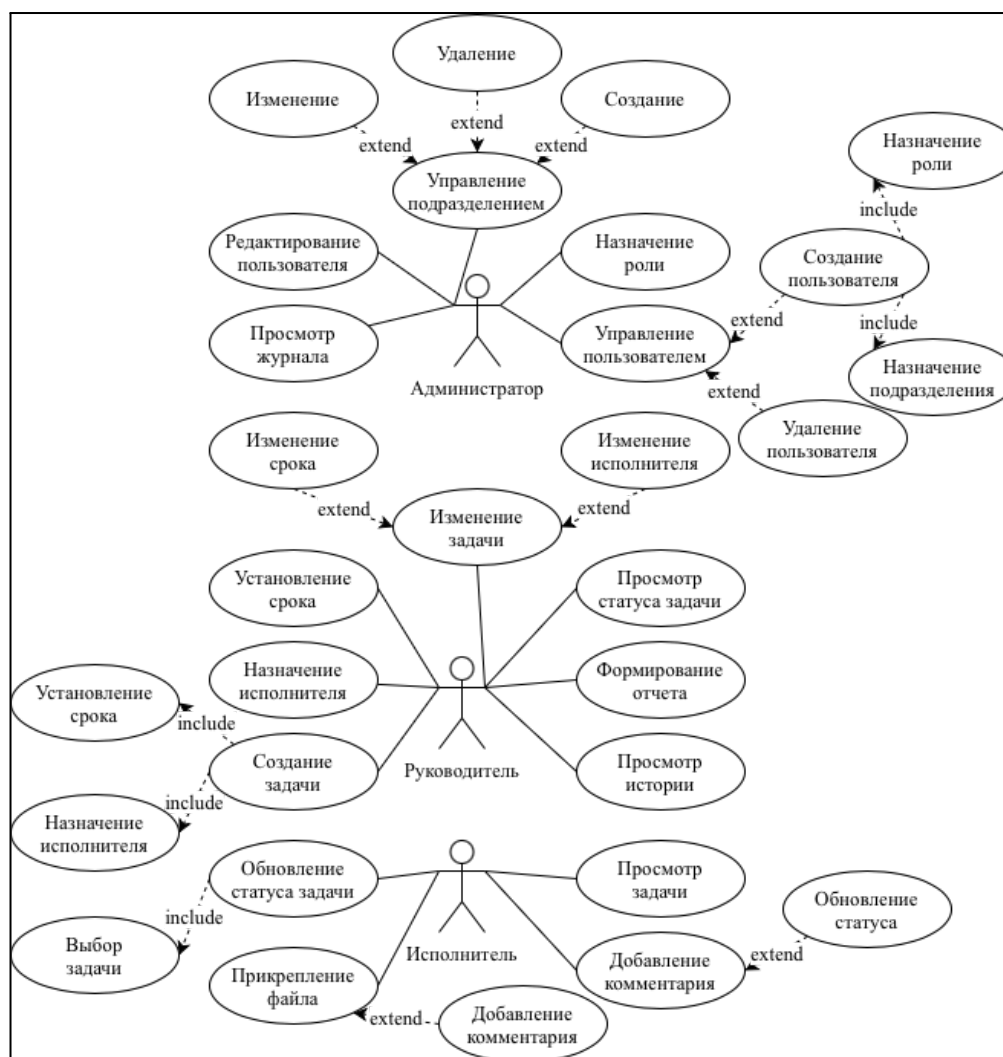


Рисунок 4.1 – UML диаграмма

Диаграмма служит промежуточным этапом между концептуальной моделью и логической схемой базы данных, обеспечивая наглядное представление объектов, их атрибутов и связей.

5 Сущности и их атрибуты

Выделение сущностей производится на основе анализа предметной области и функциональных требований системы. Каждая сущность отражает отдельный объект реального бизнес-процесса и характеризуется набором атрибутов.

1. Пользователь (User) – учетная запись сотрудника системы;
2. Роль (Role) – уровень доступа пользователя;
3. Подразделение (Department) – организационная единица;

4. Задача (Task) – рабочее задание;
5. Статус (Status) – состояние выполнения задачи;
6. Комментарий (Comment) – сообщение к задаче;
7. Вложение (Attachment) – файл, прикрепленный к задаче;
8. История изменений (TaskHistory) – журнал изменений задачи.

Атрибуты сущностей:

User

1. user_id (PK) – уникальный идентификатор пользователя;
2. full_name – ФИО пользователя;
3. email – электронная почта;
4. password – пароль для входа;
5. role_id (FK) – ссылка на роль;
6. department_id (FK) – ссылка на подразделение.

Role

1. role_id (PK) – уникальный идентификатор роли;
2. role_name – название роли;
3. description – описание роли.

Department

1. department_id (PK) – уникальный идентификатор подразделения;
2. department_name – название подразделения;
3. parent_department_id (FK) – родительское подразделение.

Task

1. task_id (PK) – уникальный идентификатор задачи;
2. title – заголовок задачи;
3. description – описание задачи;
4. creator_id (FK – User) – создатель задачи;
5. executor_id (FK – User) – исполнитель задачи;
6. status_id (FK) – текущий статус;
7. priority – уровень приоритета;
8. start_date – дата начала;
9. due_date – срок выполнения;
10. created_at – дата создания.

Status

1. status_id (PK) – уникальный идентификатор статуса;
2. status_name – название статуса.

Comment

1. comment_id (PK) – уникальный идентификатор комментария;
2. task_id (FK) – ссылка на задачу;
3. user_id (FK) – автор комментария;
4. comment_text – текст комментария;
5. created_at – дата создания.

Attachment

1. attachment_id (PK) – уникальный идентификатор вложения;
2. task_id (FK) – ссылка на задачу;
3. file_name – имя файла;

4. file_path – путь к файлу;
5. uploaded_at – дата загрузки.

TaskHistory

1. history_id (PK) – уникальный идентификатор записи;
2. task_id (FK) – ссылка на задачу;
3. old_status_id – предыдущий статус;
4. new_status_id – новый статус;
5. changed_by – пользователь, изменивший статус;
6. changed_at – дата изменения.

Атрибутный состав определен таким образом, чтобы обеспечить полноту хранения информации при сохранении нормализованной структуры. Использование первичных и внешних ключей позволяет формализовать идентификацию записей и установить логические связи между таблицами.

Связи между сущностями отражают реальные организационные отношения: назначение исполнителя, принадлежность к подразделению, изменение статуса задачи и т.д. Корректное определение типов связей (один-к-одному, один-ко-многим) обеспечивает целостность данных и предотвращает логические противоречия.

Внешние ключи реализуют механизм ссылочной целостности, исключая возможность появления «висячих» записей и обеспечивая согласованность структуры.

6 Преобразование сущностей в таблицы

Переход от концептуальной модели к реляционной осуществляется посредством отображения каждой сущности в отдельную таблицу. Атрибуты становятся столбцами таблиц, а идентификаторы — первичными ключами.

Связи реализуются через механизм внешних ключей. Иерархические структуры (например, подразделения) моделируются при помощи само ссылочных ограничений, что позволяет корректно представлять организационную структуру в рамках реляционной модели.

7 Логическая схема базы данных

Логическая схема объединяет все таблицы и их взаимосвязи в единую структуру. На данном этапе уточняются типы данных, ограничения целостности и ключевые зависимости. Логическая схема базы данных представлена на рисунке 7.1.

Разработка логической схемы позволяет систематизировать результаты предыдущих этапов проектирования и представить их в формализованном виде, пригодном для последующей физической реализации в выбранной СУБД. В рамках данной схемы фиксируются первичные и внешние ключи, определяются правила каскадного обновления и удаления, а также устанавливаются ограничения NOT NULL, UNIQUE и другие механизмы обеспечения целостности данных. Логическая модель служит основой для

оценки корректности структуры, анализа возможных избыточностей и проверки согласованности всех взаимосвязей между таблицами, что гарантирует устойчивость базы данных при эксплуатации и расширении функциональности системы.

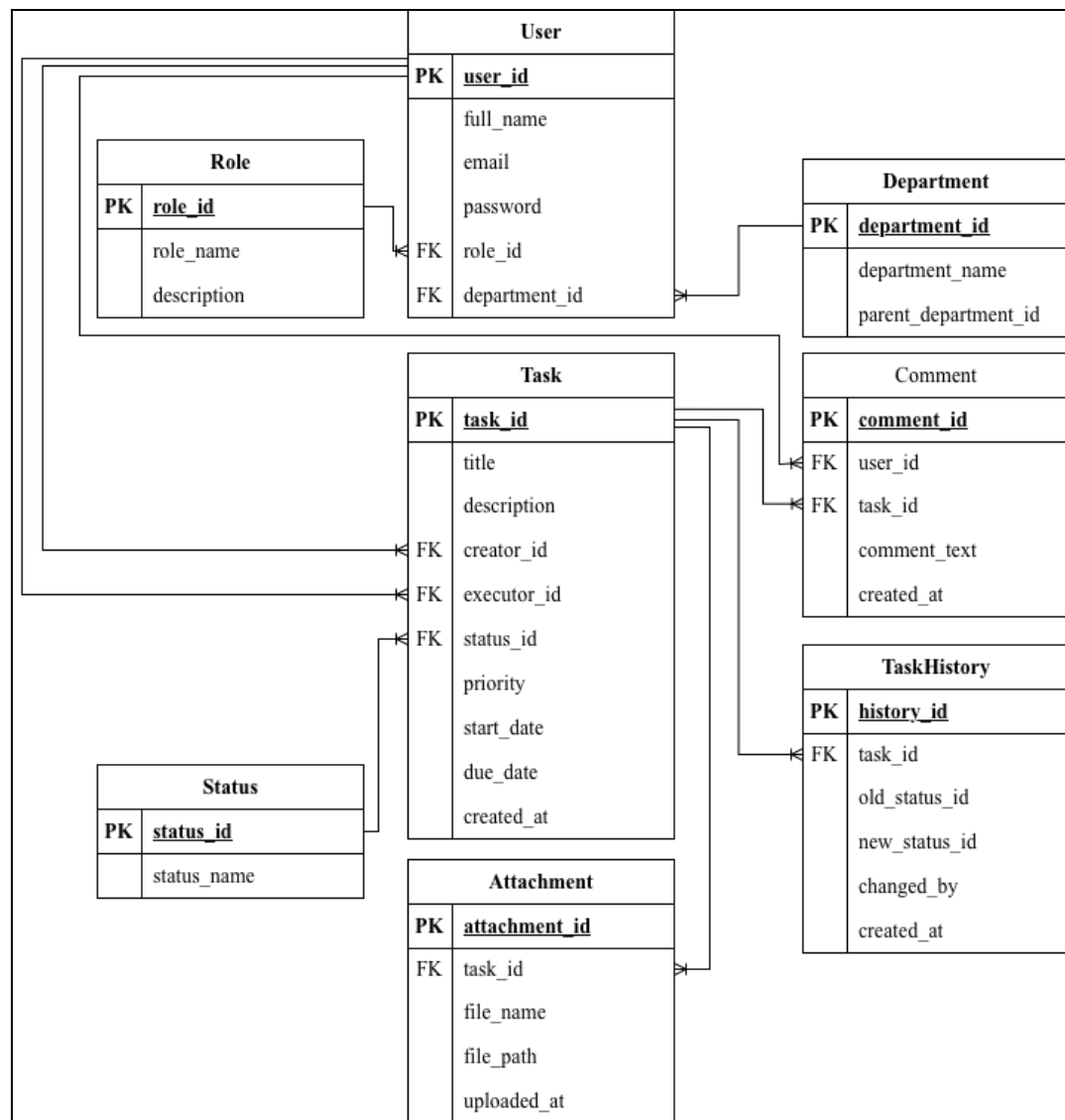


Рисунок 7.1 – Логическая схема базы данных

Схема отражает итоговую структуру хранения информации и является основой для физической реализации в конкретной СУБД. Она обеспечивает формальную завершенность проектирования на уровне логической модели.

8 Проверка нормальных форм

Анализ нормализации позволяет оценить корректность структурирования данных. Последовательная проверка соответствия 1НФ–4НФ подтверждает отсутствие избыточности, частичных и транзитивных зависимостей, а также многозначных зависимостей.

1НФ

1. Все атрибуты атомарны.
2. Повторяющихся групп нет.

2НФ

1. Все таблицы имеют простой первичный ключ.
2. Частичных зависимостей нет.

3НФ

1. Нет транзитивных зависимостей.
2. Атрибуты зависят только от РК.

4НФ

1. Многозначные зависимости отсутствуют.
2. Комментарии и вложения вынесены в отдельные таблицы.

Нормализация обеспечивает логическую непротиворечивость модели, снижает риск аномалий вставки, обновления и удаления данных и формирует устойчивую основу для масштабирования системы.

9 Денормализация

Несмотря на нормализованную структуру, в ряде случаев допускается контролируемая денормализация для повышения производительности. Такие решения принимаются на этапе оптимизации после анализа нагрузки.

Потенциальные варианты денормализации направлены на сокращение количества соединений таблиц при выполнении запросов и ускорение формирования отчетности. При этом приоритетом остается сохранение целостности данных.

10 Оценка сроков проекта

Планирование сроков разработки является неотъемлемой частью управления программным проектом и напрямую влияет на его успешную реализацию. Определение временных рамок позволяет сформировать реалистичный график выполнения работ, согласовать ожидания заказчика и команды разработки, а также обеспечить контроль достижения промежуточных результатов. При отсутствии детального планирования возрастает вероятность превышения сроков, неравномерной загрузки специалистов и снижения качества конечного продукта.

Оценка сроков выполняется с учётом объёма функциональных требований, сложности проектируемой архитектуры, уровня детализации бизнес-логики и предполагаемого объема программной реализации. Важным фактором является также взаимосвязанность этапов разработки: завершение одного этапа создает основу для начала следующего, что формирует логическую последовательность выполнения работ. При этом необходимо учитывать возможные риски, связанные с доработкой требований, исправлением выявленных ошибок и проведением дополнительного тестирования.

Оценка сроков разработки основана на декомпозиции проекта на этапы и определении трудоемкости каждого из них. Такой подход позволяет планировать ресурсы и контролировать выполнение работ. Примерные сроки работ представлены в таблице 10.1.

Таблица 10.1— Сроки разработки

Этап	Срок
Анализ требований	3 дня
Проектирование БД	4 дня
Реализация БД	5 дней
Разработка backend	10 дней
Разработка frontend	10 дней
Тестирование	5 дней

Общий срок: ~ 5–6 недель. Представленная таблица отражает последовательность выполнения задач от анализа требований до тестирования и демонстрирует реалистичный временной интервал реализации проекта.

11 Требуемые ресурсы

Реализация проекта требует четкого распределения обязанностей между специалистами различного профиля, что обеспечивает последовательность выполнения этапов разработки и контроль качества. В составе команды предусмотрены backend-разработчик, отвечающий за реализацию серверной логики, разработку REST API и взаимодействие с реляционной СУБД (SQL Server или Oracle); frontend-разработчик, реализующий пользовательский web-интерфейс и обеспечивающий корректное отображение данных; администратор базы данных (DBA), выполняющий настройку, администрирование, резервное копирование и обеспечение безопасности СУБД; а также тестировщик, осуществляющий функциональное и регрессионное тестирование системы. Для выполнения своих задач каждому специалисту необходим персональный ноутбук или рабочая станция с доступом к сети Интернет, а также доступ к серверной инфраструктуре, базе данных и тестовой среде. Данное техническое оснащение обеспечивает стабильную разработку, интеграцию клиентской и серверной частей и эффективное сопровождение проекта.

Заключение

В ходе выполнения лабораторной работы была проведена комплексная проработка предметной области, связанной с организацией контроля исполнения задач в рамках интернет-приложения. На основе анализа функциональных требований сформирована структурированная модель данных, отражающая ключевые объекты системы и их взаимосвязи.

Были определены основные сущности: пользователи, роли, подразделения, задачи, статусы, комментарии, вложения и история изменений. Для каждой сущности выделены атрибуты, определены первичные и внешние ключи, что позволило формализовать логические связи и обеспечить ссылочную целостность. Разработанная UML-диаграмма наглядно отразила структуру предметной области и стала промежуточным этапом между концептуальной и логической моделью.

Преобразование сущностей в реляционные таблицы выполнено с соблюдением принципов нормализации. Проведенный анализ подтвердил соответствие структуры первой, второй, третьей и четвертой нормальным формам. Это свидетельствует об отсутствии избыточных зависимостей, транзитивных связей и многозначных зависимостей, что обеспечивает устойчивость модели к аномалиям вставки, обновления и удаления данных.

Разработанная логическая схема обладает достаточной гибкостью и может быть масштабирована при расширении функциональности системы. При необходимости возможна контролируемая денормализация для оптимизации производительности при высоких нагрузках. Модель является корректной основой для дальнейшей физической реализации в СУБД (SQL Server, Oracle) и разработки серверной и клиентской частей приложения.

Таким образом, поставленная цель лабораторной работы достигнута: спроектирована целостная, логически обоснованная и нормализованная база данных интернет-приложения «Контроль исполнения», отвечающая требованиям надежности, структурной прозрачности и расширяемости.